

P-1 DI & Injection

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>SpringDIProject</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>

    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>6.1.8</version>
    </dependency>

  </dependencies>

</project>
```

Main.java

```
package org.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(
                AppConfig.class
            );
    }
}
```

```
        MessageController controller =
            context.getBean(
                MessageController.class
            );

        controller.displayMessage();
    }
}
```

MessageController.java

```
package org.example;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class MessageController {

    @Autowired
    private MessageService messageService;

    public void displayMessage() {

        System.out.println(
            messageService.getMessage()
        );
    }
}
```

MessageService.java

```
package org.example;

import org.springframework.stereotype.Component;

@Component
public class MessageService {

    public String getMessage() {

        return "Hello from FIRST PROJECT Service";
    }
}
```

AppConfig.java

```
package org.example;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("org.example")
public class AppConfig {

}
```

P-2: Bean Factory and XML Configuration1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>SpringXMLDemo</artifactId>
  <version>1.0</version>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>6.2.6</version>
    </dependency>
  </dependencies>

</project>
```

2. EmailService.java

```
package com.example;

public class EmailService {

    public void sendEmail() {
        System.out.println("Email Sent Successfully.");
    }

}
```

3. CustomerService.java

```
package com.example;

public class CustomerService {

    private EmailService emailService;

    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }

    public void registerCustomer() {
        System.out.println("Customer Registered Successfully.");
        emailService.sendEmail();
    }
}
```

4. beans.xml

 src/main/resources/beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="emailService"
          class="com.example.EmailService"/>

    <bean id="customerService"
          class="com.example.CustomerService">

        <property name="emailService"
                  ref="emailService"/>

    </bean>

</beans>
```

5. Main.java

```
package com.example;

import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```

public class Main {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("beans.xml");

        CustomerService customer =
            context.getBean("customerService", CustomerService.class);

        customer.registerCustomer();

        context.close();
    }
}

```

Practical No. 3: Component-Based Application Development

1. pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>ComponentDemo</artifactId>
    <version>1.0</version>

    <dependencies>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>6.2.6</version>
        </dependency>
    </dependencies>

</project>

```

2. StudentService.java

src/main/java/org/example/service/StudentService.java

```
package org.example.service;

import org.springframework.stereotype.Service;

@Service
public class StudentService {

    public String getStudentDetails() {

        return ""
            Student ID : 101
            Student Name : Rahul
            Course : MCA
            Semester : 3
            "",
    }
}
```

3. StudentController.java

src/main/java/org/example/controller/StudentController.java

```
package org.example.controller;

import org.example.service.StudentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;

@Controller
public class StudentController {

    @Autowired
    StudentService studentService;

    public void displayStudent() {

        System.out.println(
            studentService.getStudentDetails()
        );
    }
}
```

4. AppConfig.java

src/main/java/org/example/config/AppConfig.java

```

package org.example.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "org.example")
public class AppConfig {

}

```

5. Main.java

 src/main/java/org/example/Main.java

```

package org.example;

import org.example.config.AppConfig;
import org.example.controller.StudentController;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        StudentController controller =
            context.getBean(StudentController.class);

        controller.displayStudent();
    }
}

```

Output

```

Student ID : 101
Student Name : Rahul
Course : MCA
Semester : 3

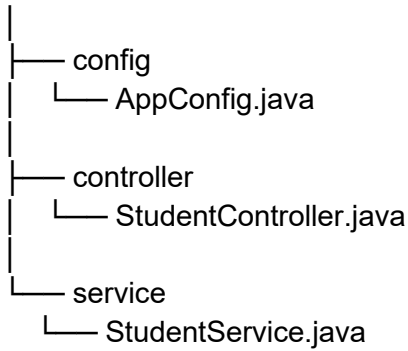
```

Important: Folder/package structure exactly આ રીતે રાખજે:

```

org.example
|
└── Main.java

```



હા Meet, **Practical No. 4: MVC Architecture** માટે કાલે આ clean અને simple practical રાખજે. તારા paste કરેલા મોટા image/base64 partની જરૂર નથી. 😊

Practical No. 4: Model-View-Controller (MVC) Architecture

Aim: Develop a Spring MVC application where the Controller handles requests, the Model stores data, and JSP pages act as Views.

1. WebConfig.java

📁 src/main/java/org/example/config/WebConfig.java

```
package org.example.config;
```

```
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ViewResolverRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
```

```
@Configuration
```

```
@EnableWebMvc
```

```
@ComponentScan(basePackages = "org.example")
```

```
public class WebConfig implements WebMvcConfigurer {
```

```
    @Override
```

```
    public void configureViewResolvers(ViewResolverRegistry registry) {
```

```
        registry.jsp("/WEB-INF/views/", ".jsp");
```

```
    }
```

```
}
```

2. HomeController.java

src/main/java/org/example/controller/HomeController.java

```
package org.example.controller;

import org.example.model.Student;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "studentForm";
    }

    @GetMapping("/studentForm")
    public String studentForm() {
        return "studentForm";
    }

    @PostMapping("/register")
    public String register(
        @RequestParam("name") String name,
        @RequestParam("course") String course,
        Model model) {

        Student student = new Student();

        student.setName(name);
        student.setCourse(course);

        model.addAttribute("student", student);

        return "result";
    }
}
```

3. Student.java

src/main/java/org/example/model/Student.java

```
package org.example.model;
```

```
public class Student {  
  
    private String name;  
    private String course;  
  
    public Student() {  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getCourse() {  
        return course;  
    }  
  
    public void setCourse(String course) {  
        this.course = course;  
    }  
}
```

4. studentForm.jsp

 src/main/webapp/WEB-INF/views/studentForm.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>
```

```
<html>
```

```
<head>
```

```
    <title>Student Registration</title>
```

```
</head>
```

```
<body>
```

```
    <h2>Student Registration Form</h2>
```

```
    <form action="register" method="post">
```

```
        Name:
```

```
        <input type="text" name="name" required>
```

```
        <br><br>
```

```
        Course:
```

```
<input type="text" name="course" required>
<br><br>

<input type="submit" value="Register">

</form>

</body>
</html>
```

5. result.jsp

📁 src/main/webapp/WEB-INF/views/result.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>

<html>
<head>
  <title>Student Details</title>
</head>

<body>

  <h2>Student Details</h2>

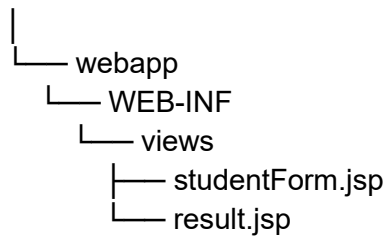
  <p>Name: ${student.name}</p>

  <p>Course: ${student.course}</p>

</body>
</html>
```

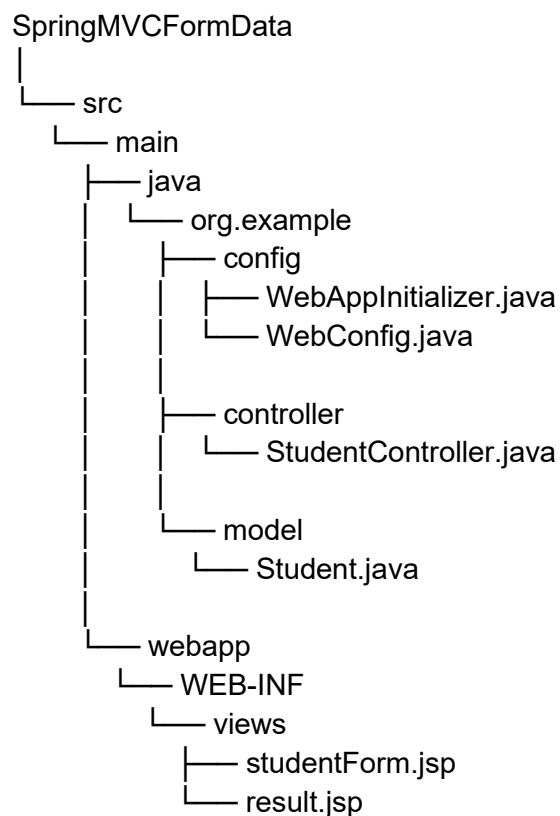
Folder Structure

```
src/main
|
├── java
│   └── org/example
│       ├── config
│       │   └── WebConfig.java
│       ├── controller
│       │   └── HomeController.java
│       └── model
│           └── Student.java
```



Haa Meet, samjhi gayo 😊 Tu je **pehla exact code aapko chhe**, ema thi j practical banavvo chhe — extra Spring Boot nathi. Toh aa rahi **same code style ma complete files**.

Project Structure



5- Reading HTML Form Data

Practical Implementation: Use Spring MVC's form binding mechanism to bind HTML form data to Java objects.

----- **WebAppInitializer.java**

📌 `src/main/java/org/example/config/WebAppInitializer.java`

```

package org.example.config;

import
org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializ
er;

public class WebAppInitializer
    extends AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return null;
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class<?>[]{
            WebConfig.class
        };
    }

    @Override
    protected String[] getServletMappings() {
        return new String[]{
            "/"
        };
    }
}

```

WebConfig.java

 [src/main/java/org/example/config/WebConfig.java](#)

```

package org.example.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ViewResolverRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
@EnableWebMvc
@ComponentScan("org.example")
public class WebConfig implements WebMvcConfigurer {

    @Override

```

```
public void configureViewResolvers(ViewResolverRegistry registry) {  
  
    registry.jsp(  
        "/WEB-INF/views/",  
        ".jsp"  
    );  
}  
}
```

Student.java

 `src/main/java/org/example/model/Student.java`

```
package org.example.model;  
  
public class Student {  
  
    private String name;  
    private String email;  
    private String course;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getEmail() {  
        return email;  
    }  
  
    public void setEmail(String email) {  
        this.email = email;  
    }  
  
    public String getCourse() {  
        return course;  
    }  
  
    public void setCourse(String course) {  
        this.course = course;  
    }  
}
```

StudentController.java

📌 `src/main/java/org/example/controller/StudentController.java`

```
package org.example.controller;

import org.example.model.Student;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class StudentController {

    @GetMapping("/")
    public String home() {
        return "redirect:/studentForm";
    }

    @GetMapping("/studentForm")
    public String showForm() {
        return "studentForm";
    }

    @PostMapping("/register")
    public String registerStudent(Student student, Model model) {

        model.addAttribute("student", student);

        return "result";
    }
}
```

📄 **studentForm.jsp**

📌 `src/main/webapp/WEB-INF/views/studentForm.jsp`

```
<html>
<head>
    <title>Student Form</title>
</head>

<body>

<h2>Student Registration Form</h2>
```

```
<form action="register" method="post">

    Name:
    <input type="text" name="name">
    <br><br>

    Email:
    <input type="email" name="email">
    <br><br>

    Course:
    <input type="text" name="course">
    <br><br>

    <input type="submit" value="Register">

</form>

</body>
</html>
```

result.jsp

 `src/main/webapp/WEB-INF/views/result.jsp`

```
<html>
<head>
    <title>Student Details</title>
</head>

<body>

<h2>Student Details</h2>

Name: ${student.name}
<br><br>

Email: ${student.email}
<br><br>

Course: ${student.course}

</body>
</html>
```

Main logic

```
public String registerStudent(Student student, Model model)
```


Aa line ma Spring MVC automatically:

HTML Form Data



Student Java Object



Model



result.jsp

`name="name" → student.name`

`name="email" → student.email`

`name="course" → student.course`

Aa **exactly tara original controller na form-binding mechanism** pramane chhe. Simple HTML form use karyu chhe, etle `showForm()` ma `new Student()` add karvani pan jarur nathi. Aa practical mate aa version perfect chhe.

Haa Meet 👍 Tame practical karva mate **tamara pehla na same Spring MVC project style** ma complete code niche aapu chu.

Practical 6: Validation and Custom Validation

Project Structure

SpringMVCValidation

|

|— pom.xml

|

|— src

|— main

|— java

|— org

|— example

|— config

```
|      | | — WebAppInitializer.java
|      | |   — WebConfig.java
|      |
|      | — controller
|      |   — StudentController.java
|      |
|      | — model
|      |   — Student.java
|      |
|      | — validator
|      |   — StudentValidator.java
|
| — webapp
|   — WEB-INF
|     — views
|       — studentForm.jsp
|       — success.jsp
```

1. WebAppInitializer.java

📁 `src/main/java/org/example/config/WebAppInitializer.java`

```
package org.example.config;
```

```
import
org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializ
er;
```

```

public class WebAppInitializer
    extends AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return null;
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class<?>[]{
            WebConfig.class
        };
    }

    @Override
    protected String[] getServletMappings() {
        return new String[]{
            "/"
        };
    }
}

```

2. WebConfig.java

 src/main/java/org/example/config/WebConfig.java

```

package org.example.config;

```

```
import org.springframework.context.annotation.ComponentScan;

import org.springframework.context.annotation.Configuration;

import org.springframework.web.servlet.config.annotation.EnableWebMvc;

import org.springframework.web.servlet.config.annotation.ViewResolverRegistry;

import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
```

```
@Configuration
```

```
@EnableWebMvc
```

```
@ComponentScan("org.example")
```

```
public class WebConfig implements WebMvcConfigurer {
```

```
    @Override
```

```
    public void configureViewResolvers(
```

```
        ViewResolverRegistry registry) {
```

```
        registry.jsp(
```

```
            "/WEB-INF/views/",
```

```
            ".jsp"
```

```
        );
```

```
    }
```

```
}
```

3. Student.java

 src/main/java/org/example/model/Student.java

```
package org.example.model;
```

```
import jakarta.validation.constraints.Email;

import jakarta.validation.constraints.NotBlank;

import jakarta.validation.constraints.Pattern;

import jakarta.validation.constraints.Size;


public class Student {

    @NotBlank(message = "Name is required")

    private String name;


    @Email(message = "Enter valid email")

    private String email;


    @Size(

        min = 6,

        max = 12,

        message = "Password must be 6-12 characters"

    )

    private String password;


    @Pattern(

        regexp = "[0-9]{10}",

        message = "Mobile should contain 10 digits"

    )

    private String mobile;
```

```
private String course;
```

```
public String getName() {  
    return name;  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
public void setEmail(String email) {  
    this.email = email;  
}
```

```
public String getPassword() {  
    return password;  
}
```

```
public void setPassword(String password) {  
    this.password = password;  
}
```

```
public String getMobile() {  
    return mobile;  
}  
  
public void setMobile(String mobile) {  
    this.mobile = mobile;  
}  
  
public String getCourse() {  
    return course;  
}  
  
public void setCourse(String course) {  
    this.course = course;  
}  
}
```

4. StudentValidator.java

 src/main/java/org/example/validator/StudentValidator.java

```
package org.example.validator;  
  
import org.example.model.Student;  
  
import org.springframework.stereotype.Component;  
import org.springframework.validation.Errors;
```

```
import org.springframework.validation.Validator;
```

```
@Component
```

```
public class StudentValidator implements Validator {
```

```
    @Override
```

```
    public boolean supports(Class<?> clazz) {
```

```
        return Student.class.equals(clazz);
```

```
    }
```

```
    @Override
```

```
    public void validate(
```

```
        Object target,
```

```
        Errors errors) {
```

```
        Student student = (Student) target;
```

```
        String course = student.getCourse();
```

```
        if (course == null ||
```

```
            !(course.equalsIgnoreCase("BCA") ||
```

```
                course.equalsIgnoreCase("MCA") ||
```

```
                course.equalsIgnoreCase("B.Tech")))) {
```

```
            errors.rejectValue(
```



```
        "course",
        "course.invalid",
        "Only BCA, MCA and B.Tech are allowed."
    );
}
}
}
```

5. StudentController.java

 src/main/java/org/example/controller/StudentController.java

```
package org.example.controller;
```

```
import jakarta.validation.Valid;
```

```
import org.example.model.Student;
```

```
import org.example.validator.StudentValidator;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Controller;
```

```
import org.springframework.validation.BindingResult;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.ModelAttribute;
```

```
import org.springframework.web.bind.annotation.PostMapping;
```

```
@Controller
```

```
public class StudentController {
```

@Autowired

private StudentValidator validator;

@GetMapping("/")

public String home() {

return "redirect:/studentForm";

}

@GetMapping("/studentForm")

public String showForm() {

return "studentForm";

}

@PostMapping("/register")

public String submitForm(

@Valid

@ModelAttribute("student")

Student student,

BindingResult result) {

validator.validate(student, result);

if (result.hasErrors()) {

```
        return "studentForm";
    }

    return "success";
}
}
```

6. studentForm.jsp

 src/main/webapp/WEB-INF/views/studentForm.jsp

```
<%@ taglib prefix="form"
    uri="http://www.springframework.org/tags/form" %>

<html>

<head>
    <title>Student Registration</title>
</head>

<body>

<h2>Student Registration Form</h2>

<form:form action="register"
    method="post"
    modelAttribute="student">
```

Name:

```
<form:input path="name"/>
```

```
<form:errors path="name"/>
```

```
<br><br>
```

Email:

```
<form:input path="email"/>
```

```
<form:errors path="email"/>
```

```
<br><br>
```

Password:

```
<form:password path="password"/>
```

```
<form:errors path="password"/>
```

```
<br><br>
```

Mobile:

```
<form:input path="mobile"/>
```

```
<form:errors path="mobile"/>
```

```
<br><br>
```

Course:

```
<form:input path="course"/>
```

```
<form:errors path="course"/>
```

```
<br><br>
```

```
<input type="submit"
      value="Register"/>
```

```
</form:form>
```

```
</body>
```

```
</html>
```

7. success.jsp

 src/main/webapp/WEB-INF/views/success.jsp

```
<html>
```

```
<head>
```

```
  <title>Success</title>
```

```
</head>
```

```
<body>
```

```
<h2>Registration Successful</h2>
```

```
</body>
```

```
</html>
```
